MLIR — A Short Primer

Ideas, architecture, and rules

Companion notes for the ARRUS NdArray graph design

2026-09-18

Contents

1 What MLIR is (and isn't) 2
- 1.1 The problem MLIR solves . . . 2
- 1.2 The one-line description . . . 2

2 Core concepts 2
- 2.1 Op (operation) . . . 2
- 2.2 Value (SSA) . . . 3
- 2.3 Type . . . 3
- 2.4 Attribute . . . 3
- 2.5 Region . . . 3
- 2.6 Block . . . 3

3 Regions: the heart of MLIR 4
- 3.1 Region kinds . . . 4
- 3.2 Region semantics are op-defined . . . 4
- 3.3 Terminators and yields . . . 4
- 3.4 Values crossing region boundaries . . . 5

4 Dialects 5
- 4.1 Some dialects worth knowing . . . 5
- 4.2 Interfaces . . . 6

5 Rewrites and progressive lowering 6
- 5.1 The rewrite discipline . . . 6
- 5.2 Progressive lowering . . . 6

6 Verification, printing, and testing 6

7 Why MLIR matters 7

8 When *not* to use MLIR 7

9 How this maps to the ARRUS NdArray design 7

10 References **8**
10.1 Official 8
10.2 Papers 8
10.3 Talks 8
10.4 Ecosystem projects (good code to read) 8
10.5 Books and long-form writing 9
10.6 Suggested reading path 9

1 What MLIR is (and isn't)

MLIR (Multi-Level Intermediate Representation) is a compiler infrastructure released by Google in 2019 and now hosted under the LLVM project. Its central idea:

> Compilation is best done as a sequence of *small*, *semantics-preserving* transformations across *multiple* intermediate representations, each at the abstraction level appropriate for the transformation.

It is **not** a single fixed IR (like LLVM IR is). It is a *framework* for building your own IR — or, more typically, several of them — and for lowering between them in a principled way. LLVM IR itself becomes one of many possible MLIR dialects.

1.1 The problem MLIR solves

Before MLIR, compiler teams routinely built ad-hoc IRs for their domain — TensorFlow had one, XLA had one, Swift had SIL, Rust had MIR, hardware design tools had theirs, and each of these had to re-invent pass management, verification, printing/parsing, rewrite patterns, testing tooling, and lowering pipelines from scratch. Each IR sat too far above LLVM IR to lower to it directly; each project paid the same infrastructure cost.

MLIR is that shared infrastructure. You define your IR by declaring dialects, and you get the whole toolchain — passes, patterns, serialization, textual round-trip, verification, testing — for free.

1.2 The one-line description

> **MLIR is SSA-based IR with user-defined dialects, ops that can nest regions of more ops, and a rewrite framework designed for progressive lowering across abstraction levels.**

Everything else in this document unpacks that sentence.

2 Core concepts

MLIR's vocabulary is small. Six terms cover most of it: **op**, **value**, **type**, **attribute**, **region**, **block**.

2.1 Op (operation)

The unit of computation. Everything is an op — arithmetic, a control flow construct, a function definition, a module, a whole compilation unit. An op has:

- a **name** (`arith.addf`, `scf.if`, `func.func`, …),
- **operands** — a list of SSA values it consumes,
- **results** — a list of SSA values it produces,
- **attributes** — a dictionary of compile-time-known data (an integer axis, a string name, a constant array, …),
- optionally, one or more **regions** (op bodies — see below),
- optionally, one or more **successors** (basic-block targets for terminator ops).

An op does not carry runtime state. It is a piece of syntax with semantics defined by its dialect.

2.2 Value (SSA)

A **value** is a typed piece of data produced by an op (or by a block's argument list). Values obey **SSA (Static Single Assignment)**: each value has exactly one point of definition; once defined, it does not change. If you want a "new" version, you produce a new value.

SSA is not just a style — it is a data-structure invariant that makes every def-use edge in the program explicit and traversable, which is what makes rewrites and dataflow analyses tractable.

2.3 Type

Every value has a **type** — a compile-time descriptor of what the value is. Types can be built-in (`i32`, `f64`, `tensor<8x128xf32>`, `memref<4xi16>`) or defined by any dialect (`!async.token`, `!linalg.range`, or, in our design, `!us4r.device_state`). Types are values-of-types, hashable, and compared structurally.

2.4 Attribute

Compile-time-known data attached to an op, indexed by string name. Attributes are plain data: integers, floats, strings, arrays, dense tensors of constants. They cannot refer to SSA values.

Use attributes for things known at compile time (an FFT axis, a convolution stride, a function name). Use operands for things computed at runtime.

2.5 Region

A region is an ordered list of **basic blocks**. It is a container of more ops. Regions are how MLIR expresses nesting: an op can carry regions, and those regions contain ops.

The critical rule: **the semantics of a region are defined by the op that owns it**. `scf.if` has two regions and picks exactly one to execute; `scf.parallel` has one region and runs it many times concurrently; `func.func` has one region and it is a function body. See §3 for the full picture.

2.6 Block

A **basic block** is a straight-line sequence of ops ending in a **terminator** op (a branch, a return, a yield). Blocks are the unit of control flow inside an SSACFG region. A block may have **block arguments** — SSA values that represent the incoming values from whichever predecessor branched to this block; block arguments replace LLVM's phi nodes.

3 Regions: the heart of MLIR

Regions are what make MLIR different from every previous production IR. It's worth being precise about them.

3.1 Region kinds

Regions come in two kinds, declared by the owning op via the `RegionKindInterface`:

SSACFG regions (the default). Contain basic blocks with control flow between them. Ops in a block execute top-to-bottom. Terminators choose the next block. This is where most computation lives.

Graph regions (no CFG). Contain a flat unordered bag of ops with no notion of before/after. Used for topology-only IRs — hardware descriptions, module namespaces, dataflow graphs whose scheduling is someone else's concern.

3.2 Region semantics are op-defined

Here is the piece that trips up newcomers: **there is no single rule about what a region *means*.** The owning op says.

Op	Regions	Semantic rule
`scf.if`	2	Exactly one region runs, chosen by the condition. Yielded value becomes the op's result.
`scf.for`	1	Region runs N times sequentially; each iteration's yields feed the next as block args.
`scf.while`	2	`before` computes a condition; if true, `after` runs; repeat.
`scf.parallel`	1	Region runs K times **in parallel** over an index space; no ordering between iterations.
`async.execute`	1	Region runs asynchronously; op yields a `!async.token` handle.
`linalg.generid`		Region is the *elementwise body*, invoked once per output element (the loop is implicit).
`func.func`	1	Region is a function body, run when the function is called.
`spirv.module`	1	Graph region: a namespace of ops with no ordering.

The same region drawing (a dashed box with ops inside) can mean mutually exclusive execution, parallel execution, iteration, or an elementwise body. The op's *documented contract* and *verifier* are what pin it down.

3.3 Terminators and yields

Every SSACFG region has a terminator at the end of each of its blocks. The terminator either transfers control (`cf.br`, `cf.cond_br`, `func.return`) or hands values *up* to the enclosing op (`scf.yield`, `linalg.yield`, `async.yield`).

The yield's operand types must match the enclosing op's result types. This is how a region communicates its result out.

3.4 Values crossing region boundaries

A region can **read** values from its enclosing scope — much like a closure captures. But whether a region *escapes* those reads to the outside depends on op semantics: `scf.if`'s region only executes if chosen; `func.func`'s region runs only when called.

A region cannot expose its **internal** ops or block arguments to the outside. Only the values it yields, via its terminator, become visible as the enclosing op's results.

4 Dialects

A **dialect** is a namespace containing related ops, types, and attributes. Every op belongs to exactly one dialect and is named `<dialect>.<op>` — `arith.addf`, `scf.if`, `linalg.generic`, `tensor.extract`, and so on.

Dialects are how MLIR expresses **multiple abstraction levels** inside one IR framework. A single MLIR module can contain ops from many dialects at once, and lowering typically means rewriting ops from one dialect into ops from a lower-level dialect.

4.1 Some dialects worth knowing

Upstream MLIR ships around 40 dialects. A representative sample:

High level:

- `func`, `arith`, `math`, `tensor`, `memref` — core numerical/data types.
- `scf` — structured control flow (`if`, `for`, `while`, `parallel`).
- `linalg` — structured linear-algebra ops with implicit loops.
- `tosa` — tensor operator set for ML compilers.
- `mhlo`, `stablehlo` — XLA-style HLO ops.
- `torch`, `torch-mlir` — PyTorch frontend.

Mid level:

- `affine` — polyhedral-friendly loops and memory access.
- `vector` — machine-independent vector ops.
- `gpu` — abstract GPU compute.
- `async` — coarse asynchrony (tasks, tokens).
- `bufferization` — turns tensor semantics into memref buffers.

Low level:

- `nvvm`, `rocdl`, `amdgpu` — vendor-specific GPU intrinsics.
- `spirv` — Vulkan/OpenGL shader IR.
- `llvm` — MLIR's mirror of LLVM IR (the classical scalar IR).
- `emitc` — emission of C source code.

Your project will typically define **its own dialects** at whatever abstraction levels it needs. That is the point.

4.2 Interfaces

Dialects avoid coupling their ops to specific concrete types via **interfaces** — think of them as C++ virtual interfaces attached to ops or types. A pass says "give me every op that implements `LoopLikeOpInterface`" and gets loops from every dialect without knowing what they are. Interfaces we care about:

- `RegionKindInterface` — SSACFG vs Graph region kind.
- `MemoryEffectsInterface` — declares an op's reads/writes/allocs.
- `SideEffectInterface` — is this op pure?
- `InferTypeOpInterface` — shape/type inference contract.
- `LoopLikeOpInterface`, `CallOpInterface`, `SymbolTable`, …

Interfaces are what let generic passes (dead-code elimination, CSE, canonicalization, licm) work across arbitrary dialects.

5 Rewrites and progressive lowering

5.1 The rewrite discipline

Passes in MLIR **never mutate** ops in place. They build new ops via a `PatternRewriter`, redirect uses with `replaceOp` / `replaceAllUsesWith`, and let dead code fall out through a DCE sweep.

This immutability is what makes rewrites composable. A pass writes a `RewritePattern` — a match/rewrite pair — and the framework applies patterns to fixpoint, invokes the verifier after each step, canonicalizes, and moves on.

Two flavors:

- **Canonicalization** — algebraic identities, `(x + 0) → x`, `broadcast(broadcast(x)) → broadcast(x)`. Runs everywhere.
- **Conversion** — cross-dialect lowering, driven by a `ConversionTarget` (which ops are legal) and a set of `ConversionPatterns`. This is how you go from `linalg` to `affine`, from `affine` to `scf`, from `scf` to `cf`, from `cf` to `llvm`.

5.2 Progressive lowering

Compilation in MLIR is normally a **staircase**: a program starts high (say, `linalg` over `tensor`) and descends stepwise (`linalg` → `affine` → `scf` → `cf` → `llvm` → LLVM IR → machine code). Each step is a small, testable transformation. Passes at each step only need to understand the two dialects they bridge, not the whole compilation.

This is the payoff of the "multi-level" in the name. Instead of one IR trying to be both close to source and close to machine (and being bad at both), you get many IRs, each fitted to its transformation.

6 Verification, printing, and testing

Every op has a **verifier** — a hand-written or generated function that checks structural invariants (result count, operand types, attribute keys, region kinds, …). The verifier runs after every pass. A miscompiled program fails fast at the pass boundary, not at runtime.

Every op has a **textual form** for printing and parsing. MLIR text is round-trippable: parse a `.mlir` file, print it back, get the same bytes. This makes textual snapshots the standard testing tool: `mlir-opt input.mlir -my-pass | FileCheck input.mlir`. Passes are tested with textual before/after fixtures.

Op definitions are generated from a declarative DSL called **ODS** (Operation Definition Specification) written in TableGen. ODS produces the C++ classes, verifiers, printers, parsers, and builder methods — you write ~30 lines of declarative spec instead of ~500 lines of C++.

7 Why MLIR matters

- **One toolchain, many IRs.** Every new dialect gets passes, patterns, verification, textual round-trip, and testing for free.
- **Progressive lowering.** A pass only needs to understand two neighbouring abstraction levels, not the whole compiler.
- **SSA + regions.** Fine-grained explicit dataflow *plus* structured control flow *plus* nesting means you can express high-level ops (a whole matmul, a whole kernel launch) without giving up SSA.
- **Interfaces over inheritance.** Generic passes work across arbitrary dialects.
- **Testable.** Every transformation has a textual before/after fixture and a verifier as a safety net.

The ML/AI compiler world converged on it fast: TensorFlow's XLA, PyTorch (via `torch-mlir`), OpenXLA, IREE, Modular Mojo, Triton (in part), CIRCT (hardware), Flang (Fortran frontend). It's now the default answer to "we need a compiler for a new domain."

8 When *not* to use MLIR

- **Small, single-domain problems** where an ad-hoc AST + one code generator is faster to build than defining dialects and passes.
- **Purely dynamic execution** with no ahead-of-time transformations (a plain interpreter has no IR to speak of).
- **Cases where you want the runtime tightly coupled to source code layout** (source-to-source translators can be simpler as tree transformers).

For anything that needs multiple abstraction levels, cross-hardware targeting, or serious optimization passes, MLIR is the answer.

9 How this maps to the ARRUS NdArray design

Our IR (`arrus::framework`) borrows MLIR's *shape* without depending on MLIR at runtime:

MLIR concept	ARRUS analogue
Op	`arrus::framework::Op`
SSA Value	`arrus::framework::Value`
Type	`arrus::framework::NdArrayType`
Attribute	`arrus::framework::Attribute`
Dialect	Namespace prefix on op names: `arrus.*`, `us4r.*`, `gpu.*`, …

MLIR concept	ARRUS analogue
RegionKindInterface	Will be an enum on our region records
ODS TableGen	Hand-written op registry entries (v1)
PatternRewriter	`Graph::copy_with` + `replaceAllUsesWith`
Verifier	Per-op verifier hook in `OpRegistry`
Progressive lowering	`Backend::compile` lowering Level-1 → Level-2

The design is intentionally MLIR-shaped so that a future migration to real MLIR is mechanical rather than a rewrite.

10 References

10.1 Official

- **MLIR home**: https://mlir.llvm.org
- **Language reference** (the definitive spec): https://mlir.llvm.org/docs/LangRef/
- **Tutorial (Toy language, 7 chapters)** — build your own frontend from scratch: https://mlir.llvm.org/docs/Tutorials/Toy/
- **Dialect list**: https://mlir.llvm.org/docs/Dialects/
- **Operation Definition Specification (ODS)**: https://mlir.llvm.org/docs/DefiningDialects/Operations/

10.2 Papers

- Chris Lattner *et al.*, **"MLIR: Scaling Compiler Infrastructure for Domain Specific Computation."** *CGO 2021.* https://arxiv.org/abs/2002.11054
- Uday Bondhugula, **"High Performance Code Generation in MLIR: An Early Case Study with GEMM."** 2020. https://arxiv.org/abs/2003.00532

10.3 Talks

- Chris Lattner, **"MLIR Primer"** (LLVM Developers Meeting 2019 keynote) — the original pitch: https://www.youtube.com/watch?v=Y4SvqTtOIDk
- **MLIR tutorial at CGO 2020**: https://www.youtube.com/watch?v=Y4SvqTtOIDk
- Various later LLVM Developers' Meeting sessions each October and April — the archive is worth scanning for "MLIR" in the title.

10.4 Ecosystem projects (good code to read)

- `torch-mlir` — PyTorch → MLIR frontend: https://github.com/llvm/torch-mlir
- **IREE** — MLIR-based end-to-end ML compiler + runtime: https://github.com/iree-org/iree
- **OpenXLA / StableHLO** — MLIR-based XLA: https://github.com/openxla/xla, https://github.com/openxla/stablehlo
- **CIRCT** — MLIR for hardware design: https://github.com/llvm/circt
- **Triton** — GPU DSL, uses MLIR internally: https://github.com/openai/triton
- **Flang** — Fortran frontend built on MLIR: https://github.com/llvm/llvm-project/tree/main/flang

10.5 Books and long-form writing

- Mehdi Amini *et al.*, **"MLIR in Action"** — Manning, in progress. Read the MEAP chapters as they land.
- Alex Zinenko's blog — the clearest short-form writing on MLIR internals: https://llvm.org/devmtg/2022-09/slides/ (his LLVM Devmtg talks) and posts under the MLIR discourse tag.
- The **MLIR Discourse forum** — the community's primary discussion venue and a good place to lurk: https://discourse.llvm.org/c/mlir/31

10.6 Suggested reading path

1. Read the "MLIR Primer" talk transcript or watch the video (~50 min).
2. Skim the LangRef sections **Operations**, **Regions**, **Blocks**, **Types**, **Attributes** (~30 min).
3. Work through the **Toy tutorial** chapters 1–3 (~2 hours). Skim 4–7.
4. Read one real dialect end-to-end. `scf` is a good target: small, familiar concepts, uses regions non-trivially.
5. Then read the MLIR CGO 2021 paper for the design rationale.

That is enough to be fluent for ARRUS's purposes.